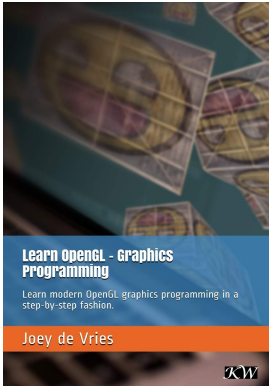


Scene Graph

Introduction to Computer Graphics - Exercise 04

Pirmin Pfeifer



► OpenGL Scene Graph Tutorial

Add a new file for the SceneGraph (SceneGraph.h and SceneGraph.cpp) to the rasterizer project and add the include to the header file:

```
#pragma once
#include <glm/glm.hpp>
#include "glm/gtc/matrix_transform.hpp"
#include <memory>
#include <vector>

#include "Mesh.h"
```

File: SceneGraph.h

Add a new file for the SceneGraph (SceneGraph.h and SceneGraph.cpp) to the rasterizer project and add the include to the header file:

```
#pragma once
#include <glm/glm.hpp>
#include "glm/gtc/matrix_transform.hpp"
#include <memory>
#include <vector>

#include "Mesh.h"
```

File: SceneGraph.h

glm is the OpenGL math library, it provides all required math types and functions.

Add a new file for the SceneGraph (SceneGraph.h and SceneGraph.cpp) to the rasterizer project and add the include to the header file:

```
#pragma once
#include <glm/glm.hpp>
#include "glm/gtc/matrix_transform.hpp"
#include <memory>
#include <vector>

#include "Mesh.h"
```

File: SceneGraph.h

glm is the OpenGL math library, it provides all required math types and functions. **glm** needs to be linked into the rasterizer.

Add a Transform struct¹:

```
struct Transform {  
    /*SPACE INFORMATION*/  
    // Local space information  
    glm::vec3 pos      = {0.0f, 0.0f, 0.0f};  
    glm::vec3 eulerRot = {0.0f, 0.0f, 0.0f};  
    glm::vec3 scale    = {1.0f, 1.0f, 1.0f};  
  
    glm::mat4 getLocalModelMatrix();  
    glm::mat4 getGlobalModelMatrix(const glm::mat4& parentMatrix);  
};
```

¹File: SceneGraph.h

Add a Transform struct¹:

```
struct Transform {  
    /*SPACE INFORMATION*/  
    // Local space information  
    glm::vec3 pos      = {0.0f, 0.0f, 0.0f};  
    glm::vec3 eulerRot = {0.0f, 0.0f, 0.0f};  
    glm::vec3 scale    = {1.0f, 1.0f, 1.0f};  
  
    glm::mat4 getLocalModelMatrix();  
    glm::mat4 getGlobalModelMatrix(const glm::mat4& parentMatrix);  
};
```

`getGlobalModelMatrix()` calculates the current local matrix by multiplying:

¹File: SceneGraph.h

Add a Transform struct¹:

```
struct Transform {  
    /*SPACE INFORMATION*/  
    // Local space information  
    glm::vec3 pos      = {0.0f, 0.0f, 0.0f};  
    glm::vec3 eulerRot = {0.0f, 0.0f, 0.0f};  
    glm::vec3 scale    = {1.0f, 1.0f, 1.0f};  
  
    glm::mat4 getLocalModelMatrix();  
    glm::mat4 getGlobalModelMatrix(const glm::mat4& parentMatrix);  
};
```

`getGlobalModelMatrix ()` calculates the current local matrix by multiplying:

► `glm:: translate ()`

¹File: SceneGraph.h

Add a Transform struct¹:

```
struct Transform {  
    /*SPACE INFORMATION*/  
    // Local space information  
    glm::vec3 pos      = {0.0f, 0.0f, 0.0f};  
    glm::vec3 eulerRot = {0.0f, 0.0f, 0.0f};  
    glm::vec3 scale    = {1.0f, 1.0f, 1.0f};  
  
    glm::mat4 getLocalModelMatrix();  
    glm::mat4 getGlobalModelMatrix(const glm::mat4& parentMatrix);  
};
```

`getGlobalModelMatrix ()` calculates the current local matrix by multiplying:

- ▶ `glm:: translate ()`
- ▶ `glm:: rotate ()` ($y \cdot x \cdot z$)

¹File: SceneGraph.h

Add a Transform struct¹:

```
struct Transform {  
    /*SPACE INFORMATION*/  
    // Local space information  
    glm::vec3 pos      = {0.0f, 0.0f, 0.0f};  
    glm::vec3 eulerRot = {0.0f, 0.0f, 0.0f};  
    glm::vec3 scale    = {1.0f, 1.0f, 1.0f};  
  
    glm::mat4 getLocalModelMatrix();  
    glm::mat4 getGlobalModelMatrix(const glm::mat4& parentMatrix);  
};
```

`getGlobalModelMatrix ()` calculates the current local matrix by multiplying:

- ▶ `glm:: translate ()`
- ▶ `glm:: rotate ()` ($y \cdot x \cdot z$)
- ▶ `glm:: scale ()`

¹File: SceneGraph.h

Transform - Implementation

For the implementation of `getLocalModelMatrix ()`

Transform - Implementation

For the implementation of `getLocalModelMatrix ()`
we just call `getGlobalModelMatrix ()`

Transform - Implementation

For the implementation of `getLocalModelMatrix ()`
we just call `getGlobalModelMatrix ()`
with the identity matrix `glm::identity <glm::mat4 >()`

Transform - Implementation

For the implementation of `getLocalModelMatrix ()`

we just call `getGlobalModelMatrix ()`

with the identity matrix `glm::identity<glm::mat4>()`

```
glm::mat4 Transform::getLocalModelMatrix()  
{  
    return getGlobalModelMatrix(glm::identity<glm::mat4>());  
}
```

File: SceneGraph.cpp

The Entity class is the node of our graph:

```
class Entity {
public:
    Transform transform = {};

    // Global space information concatenate in matrix
    glm::mat4 modelMatrix = glm::mat4(1.0f);

    Entity*          parent    = nullptr;
    std::vector<std::unique_ptr<Entity>> children = {};

    std::vector<std::shared_ptr<Mesh>> meshes = {};

    // constructor.
    Entity(const std::vector<std::shared_ptr<Mesh>> meshes = {});

    // add new child with std::make_unique and set this as parent
    void      addChild(const std::vector<std::shared_ptr<Mesh>> meshes = {});

    // update self and child modelMatrix
    void updateSelfAndChild();
};
```

File: SceneGraph.h

Entity - Implementation

We use `std::make_unique < Entity >(meshes)` to create a new unique Entity object on the heap:

We use `std::make_unique<Entity>(meshes)` to create a new unique Entity object on the heap:

```
void Entity::addChild(const std::vector<std::shared_ptr<Mesh>> meshes)
{
    children.emplace_back(std::make_unique<Entity>(meshes));
    children.back()->parent = this;
}
```

File: SceneGraph.cpp

Entity - Implementation

`std::updateSelfAndChild ()` recursively updates our
modelTransform:

Entity - Implementation

`std::updateSelfAndChild()` recursively updates our `modelTransform`:

```
void Entity::updateSelfAndChild()
{
    if (parent) {

    } else {

    }
}
```

```
}
```

`std::updateSelfAndChild()` recursively updates our `modelTransform`:

```
void Entity::updateSelfAndChild()
{
    if (parent) {
        modelMatrix = transform.getGlobalModelMatrix(parent->modelMatrix);
    } else {
```

```
}
```

```
}
```

`std::updateSelfAndChild()` recursively updates our `modelTransform`:

```
void Entity::updateSelfAndChild()
{
    if (parent) {
        modelMatrix = transform.getGlobalModelMatrix(parent->modelMatrix);
    } else {
        // for the root node the global matrix is the same as the local matrix
        modelMatrix = transform.getLocalModelMatrix();
    }
}
```

```
}
```

`std::updateSelfAndChild()` recursively updates our `modelTransform`:

```
void Entity::updateSelfAndChild()
{
    if (parent) {
        modelMatrix = transform.getGlobalModelMatrix(parent->modelMatrix);
    } else {
        // for the root node the global matrix is the same as the local matrix
        modelMatrix = transform.getLocalModelMatrix();
    }

    // recursively update children
    // so they incorporate the new model matrix of the parent
    for (auto& child : children) {
        child->updateSelfAndChild();
    }
}
```

File: SceneGraph.cpp

The shader need a new function²:

```
void setMat4x4 ( const std::string & name, const glm::mat4x4 & value) const;  
    glUniformMatrix4fv ( glGetUniformLocation (ID, name.c_str ()), 1, GL_FALSE , glm::value_ptr (value));
```

File: Shader.cpp

²We need to include "glm/ext.hpp" in the shader.cpp

The shader need a new function²:

```
void setMat4x4 ( const std::string & name, const glm::mat4x4 & value) const;  
    glUniformMatrix4fv ( glGetUniformLocation (ID, name.c_str ()), 1, GL_FALSE , glm::value_ptr (value));
```

File: Shader.cpp

The default.vert shader also need some adjustment to use our new:

```
uniform mat4x4 model;
```

²We need to include "glm/ext.hpp" in the shader.cpp

The shader need a new function²:

```
void setMat4x4 (const std::string & name, const glm::mat4x4 & value) const;  
glUniformMatrix4fv ( glGetUniformLocation (ID, name.c_str (), 1, GL_FALSE , glm::value_ptr (value));
```

File: Shader.cpp

The default.vert shader also need some adjustment to use our new:

```
uniform mat4x4 model;  
gl_Position = model * vec4(aPos.xyz, 1.0);
```

File: default.vert

²We need to include "glm/ext.hpp" in the shader.cpp

Update main Function

Create a SceneGraph (you can go wild with the content):

```
Entity root;  
root.addChild({suzan});  
root.children[0]->transform.scale = {0.5f, 0.5f, 0.5f};
```

File: main.cpp

Update main Function

Create a SceneGraph (you can go wild with the content):

```
Entity root;  
root.addChild({suzan});  
root.children[0]->transform.scale = {0.5f, 0.5f, 0.5f};
```

File: main.cpp

We also need to update the draw logic to traverse the newly build SceneGraph!

Update Render Loop

We can use different strategies to traverse the scene graph:
This example depicts a depth first search without recursion:

Update Render Loop

We can use different strategies to traverse the scene graph:
This example depicts a depth first search without recursion:

```
root.updateSelfAndChild();
```

Update Render Loop

We can use different strategies to traverse the scene graph:
This example depicts a depth first search without recursion:

```
root.updateSelfAndChild();  
  
// draw  
std::vector<Entity*> frontier = { &root };
```

Update Render Loop

We can use different strategies to traverse the scene graph:
This example depicts a depth first search without recursion:

```
root.updateSelfAndChild();  
  
// draw  
std::vector<Entity*> frontier = { &root };  
while (!frontier.empty())  
{
```

```
    }
```

```
    }
```

Update Render Loop

We can use different strategies to traverse the scene graph:
This example depicts a depth first search without recursion:

```
root.updateSelfAndChild();  
  
// draw  
std::vector<Entity*> frontier = { &root };  
while (!frontier.empty())  
{  
    const auto currentEntity = frontier.back();
```

```
    }
```

```
}  
}
```


Update Render Loop

We can use different strategies to traverse the scene graph:
This example depicts a depth first search without recursion:

```
root.updateSelfAndChild();  
  
// draw  
std::vector<Entity*> frontier = { &root };  
while (!frontier.empty())  
{  
    const auto currentEntity = frontier.back();  
    frontier.pop_back();  
    for (const auto& child : currentEntity->children)  
    {  
        frontier.push_back(child.get());  
    }  
}
```

```
    }  
}
```

Update Render Loop

We can use different strategies to traverse the scene graph:
This example depicts a depth first search without recursion:

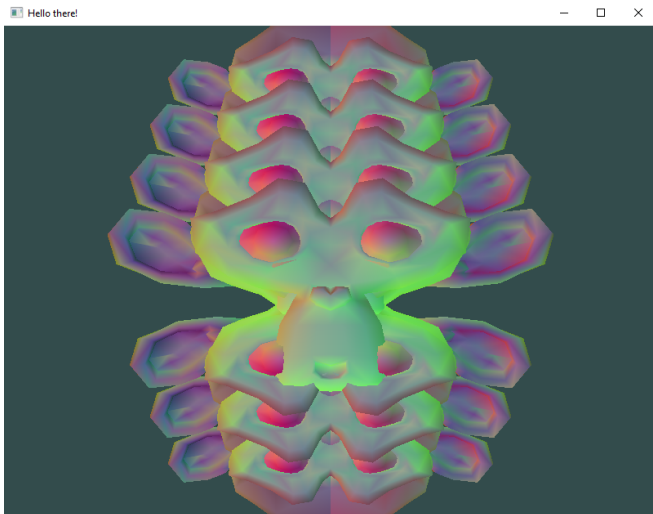
```
root.updateSelfAndChild();

// draw
std::vector<Entity*> frontier = { &root };
while (!frontier.empty())
{
    const auto currentEntity = frontier.back();
    frontier.pop_back();
    for (const auto& child : currentEntity->children)
    {
        frontier.push_back(child.get());
    }

    defaultShader.setMat4x4("model", currentEntity->modelMatrix);
    for (const auto& mesh : currentEntity->meshes) {
        mesh->Draw();
    }
}
```

File: main.cpp (render loop)

Rendered Scene Graph



Now it is your turn!